数字逻辑基础与Verilog设计
---基础篇

第2章 逻辑电路导论

本章内容

- 逻辑变量与函数及其表示（2.1-2.4）
- 布尔代数（2.5）
- 利用门电路进行综合（2.6-2.8）
- 函数化简与卡诺图（2.11-2.14）
- **FPGA及HDL语言介绍（2.9-2.10）**

5. FPGA及HDL语言介绍

- **FPGA：Field Programmable Gate Array现场可编程门阵列**
- **可编程逻辑器件（如CPLD、FPGA）可以通过软件编程而对其硬件结构和工作方式进行重构，使硬件的设计可以如同软件设计那样方便快捷。改变了传统的数字系统设计方法、设计过程和设计观念，促进了EDA技术迅速发展。**

5. FPGA及HDL语言介绍

- **ASIC、FPGA及CPLD的区别：**
  - ASIC是定制IC，内部功能已固化，只能开启或关闭不同的功能模块。而FPGA、CPLD是可编程器件，用户可根据自己的功能需求进行开发设计，为我们开展数字逻辑、组成原理、体系结构等课程实验提供绝佳硬件平台。
  - CPLD是基于ROM结构，下电后代码不丢失，而FPGA是基于RAM结构，下电后代码丢失。而且由于工艺差别，CPLD逻辑资源不可能做的太大，FPGA能够做到数万乃至百万逻辑门。

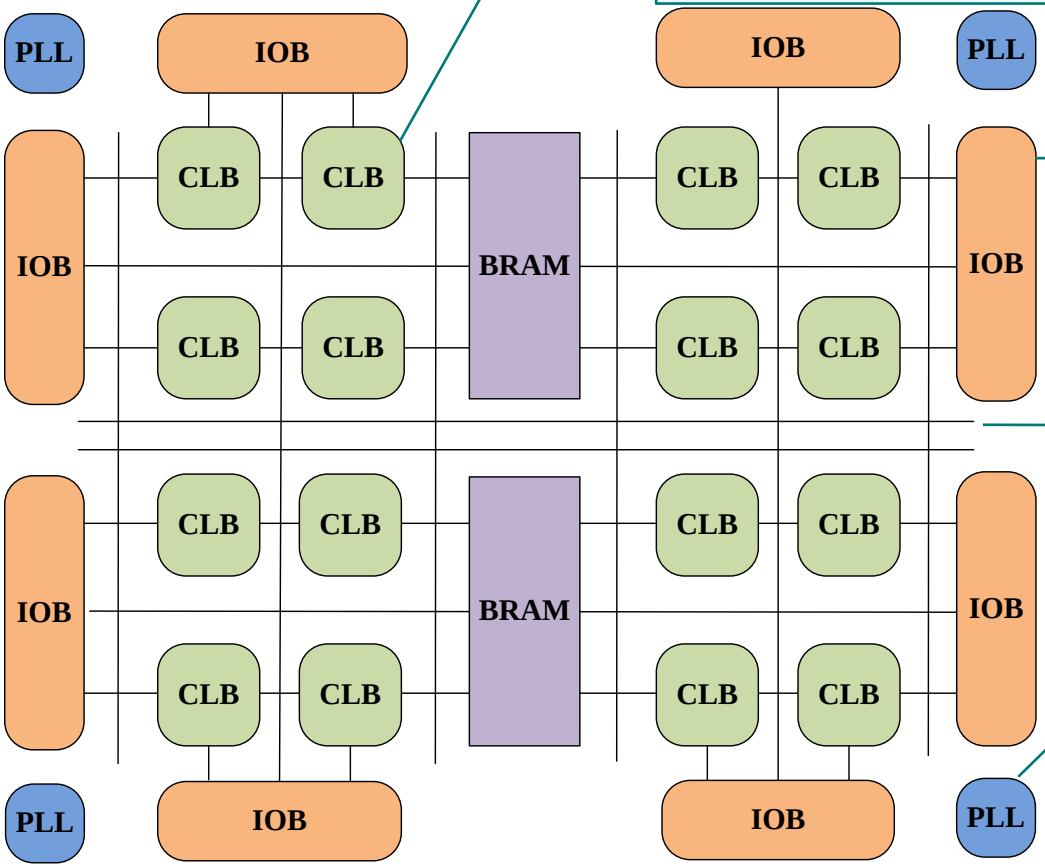
5. FPGA及HDL语言介绍

- 两大FPGA主流厂商：Xilinx 和Altera
  - 欧洲使用Xilinx的人多一些，亚太地区使用Altera的多一些。
  - 两家公司的可编程逻辑器件占到全球市场的60%以上
  - FPGA开发设计的基本理论都一样。只是一些“名词”不同而已。

5. FPGA及HDL语言介绍

- **FPGA的基本结构：**

⑩可配置逻辑块CLB（Configurable Logic Block）
- ◆ 查找表（look up table，LUT）
- ◆ 触发器（FF）
- ◆多路选择器（MUX）



⑩输出输入块IOB（Input output Block）
- ◆ FPGA外部器件的物理接口

⑩内部连线（Interconnect）
- ◆ 将内部的相关逻辑进行互联
- ◆ 起点和终点均为IOB

⑩其他一些模块：如BRAM、PLL（锁相环）、乘法器等

5. FPGA及HDL语言介绍

- **Nexys 4 DDR 板载 FPGA：Xilinx ARTIX – 7 FPGA**

XC7A100T-1CSG324C

Artix – 7 100T 产品特性：

- **15,850 片逻辑片，每片包含4个6输入查找表和8个触发器**
- **4,860 千比特高速块RAM**
- **6个带有锁相环(PLL)的时钟管理模块**
- **240个数字信号处理单元**
- **超过450 MHz 的内置时钟速率**
- **片上模数转换器(XADC)**

5. FPGA及HDL语言介绍

FPGA的一般设计流程

1.电路设计

2.代码编写

3.功能仿真

4.综合实现

5.上板调试

FPGA的一般设计流程

1.电路设计

- 根据需求指定电路设计方案。比如设计一个时序电路，需要明确系统功能，决定采用同步还是异步，数据通路和控制电路如何设计，需要几个触发器，电路状态如何转换，等等。
- 通常，电路设计细化到寄存器传输级（RTL）即可，无需精确到门级。

FPGA的一般设计流程

2.代码编写

- 硬件描述语言是对硬件电路进行行为描述、寄存器传输描述或者结构化描述的一种语言，常用两种语言：VHDL（V-超高速IC）与Verilog HDL;
- HDL即Hardware Description Language，是对电路的描述，而非Design。本课程采用Verilog HDL语言
- 可层次化设计，通用性强，适合于复杂逻辑电路设计。也可以调用一些通用IP核。

FPGA的一般设计流程

3.功能仿真

- 通过软件仿真的方式验证电路是否符合最初设计需求。
- 不涉及任何电路的延迟，仅验证代码所实现的基本功能。
- 仿真代码给定输入信号的逻辑值，仿真器以时序图的形式给出相应输出。
- 仿真工具首推ModelTech公司的ModelSim，也可以用Vivado自带功能仿真器。

FPGA的一般设计流程

4.综合实现

- 完成HDL代码（较高层次电路描述）到真实芯片电路的转换过程，类似于编译器将高级语言转换为目标机器的二进制代码的过程。
- 综合阶段：网表生成：检查代码语法，给出错误报告。将代码编译为由基本逻辑单元连接而成的逻辑网表。门级优化：将用户的设计自动转换成一个等价或更优电路；工艺映射：决定网表中的每一个元件如何在FPGA的可用资源中实现。

FPGA的一般设计流程

4.综合实现（接）

- 实现阶段：将综合出的网表映射为FPGA芯片内部固有的硬件逻辑模块上（称为布局），然后基于布局的拓扑，利用内部的连线资源，将逻辑模块连接起来（称为布线）。
- 时序分析：计算所有模块和走线延迟，产生一系列表格以量化电路性能。并用来确定生成电路是否满足时序要求。
- 若无异常，将生成一个bitstream比特流文件，该文件即为最终电路。
- Xilinx公司FPGA综合工具：Vivado&ISE

FPGA的一般设计流程

5.上板调试

- 将综合实现生成的比特流文件下载到FPGA芯片中，运行电路，观察其工作是否正常，若有问题，需要调试定位出错的原因。

5. FPGA及HDL语言介绍

什么是硬件描述语言HDL？

- 具有特殊结构能够对硬件逻辑电路的功能进行描述的一种高级编程语言；
- 这种特殊结构能够：
  - 描述电路的连接
  - 描述电路的功能
  - 在不同抽象级上描述电路
  - 描述电路的时序
  - 表达具有并行性
- 使用HDL，甚至可以自己编写一个自己的单片机出来，然后在FPGA上实现！

5. FPGA及HDL语言介绍

HDL主要有两种：Verilog和VHDL

- **VHDL出现较晚，由美国国防部开发，1987年称为IEEE标准。起源于ADA语言，格式严谨，不易学习。**
- **Verilog起源于C语言，因此非常类似于C语言, 易掌握。 1995年成为IEEE标准。**
- **目前在美国，高层次数字系统设计领域中，应用Verilog和VHDL的比率是80％和20％；日本和台湾和美国差不多；而在欧洲VHDL发展的比较好。在中国很多集成电路设计公司都采用Verilog，但VHDL也有一定的市场。**

Verilog HDL基础

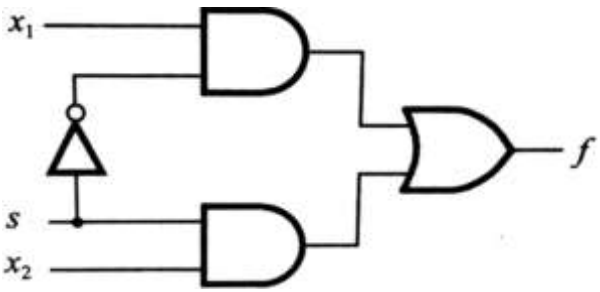
- 本节课知识要点：
  - Verilog HDL设计模块的基本结构。
  - 层次化Verilog编码

5. FPGA及HDL语言介绍

Verilog 设计模块的基本结构

设计模块

- 模块端口定义
- 模块内容
  - I/O说明
  - 信号类型说明
  - 功能描述




```
module eg1(x1,x2,s,f);
    input    x1,x2,s;
    output   f;

    not (k, s);
    and (g, k, x1);
    and (h, s, x2);
    or (f, g, h);
endmodule
```

5. FPGA及HDL语言介绍

(1)模块（端口）定义

- 模块定义格式如下:

```
module 模块名（端口1，端口2，端口3，…）；
……
endmodule
```

- 模块端口定义用来声明设计电路模块的输入、输出端口，在端口定义的圆括弧中，是设计电路模块与外界联系的全部输入输出端口信号或引脚，它是设计实体对外的一个通信界面，多个端口名之间用“，”分隔。例如，

```
module adder（sum，cont，ina，inb，cin）；
```

5. FPGA及HDL语言介绍

(2) 模块内容

– 模块I/O说明
- 声明端口定义中各端口数据流方向，包括输入(input)、输出(output)和双向(inout)。
- 格式如下：input 端口1，端口2，…;

– 信号类型声明
- 用来说明设计电路的功能描述中，所用的信号的数据类型以及函数声明。
- 数据类型有连线(wire)、寄存器(reg)、整型(integer)、实型(real)和时间(time)等类型。

– 功能描述(建模)
- 用来描述输入、输出端口间的逻辑功能，在电路上相当于器件的内部电路结构。
- 可用门级结构（包括元件例化instantiate）、数据流方式（assign）、行为级描述方式（always块、initial块）来实现。

(2) 模块内容

- 信号类型声明
  - **wire**，意为连线型，一般用于组合逻辑中。**Verilog**模块中的输入、输出信号默认类型为**wire**型。 **wire**型数据通常用**assign**语句赋值。
  - **reg**，意为寄存器型，既可以用组合逻辑实现，亦可用于时序逻辑中，即用触发器实现。**reg**型数据必须被放在过程语句中（如**initial**、**always**）中，通过过程赋值语句赋值。

另一种模块声明方式（推荐）：

```
module bottom（
    input  wire a,
    input  wire b,
    output  wire y,
    output reg z
）；
    ......
endmodule
```

课件与教材对此不做区分，推荐采用第二种方式。

5. FPGA及HDL语言介绍

- 功能描述

（1）用门级原始结构语句建模

– 结构描述是一种低层次的描述方法。是对设计电路的结构进行描述，即描述设计电路使用的元件及这些元件间的连接关系。
– 门电路的关键词为：**and**、**or**、**not**、**nand**、**nor**、**xor**、**nxor**等
– 格式：逻辑门类型 <实例名称（可选）> (输出端口，输入端口);
– 元件例化方式建模是利用**Verilog**提供的元件库中的元件来实现
– 例**1**：用与门例化元件定义一个**3**输入端与门

```
and myand3(y, a, b, c);
/* 在本模块中调用了一个与门，实例名称为myand3，输入信号为a，b，c；输出为y。*/
```

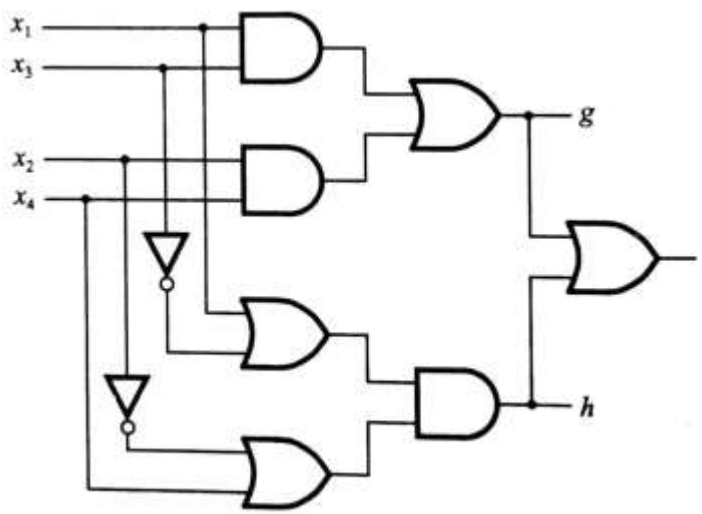
5. FPGA及HDL语言介绍

- 功能描述

（1）用门级原始结构语句建模

– 若缺省实例名称，则内部会自动分配一个实例名称；

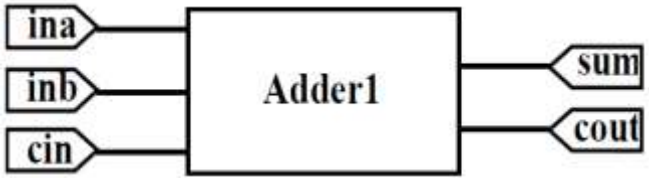
– 例2：门级语句对下图进行建模。




```
module eg2(x1,x2,x3,x4,f,g,h);
    input   x1,x2,x3,x4;
    output  f, g, h;
    and (z1,x1,x3);
    and (z2,x2,x4);
    or (g,z1,z2);
    or (z3,x1,~x3);
    or (z4,x4,~x2);
    and (h,z3,z4);
    or (f,h,g);
endmodule
```

（2）数据流描述方式：用assign语句建模

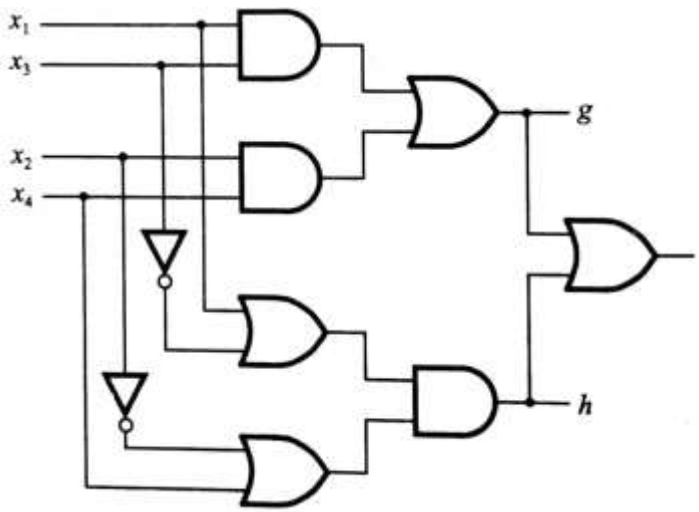
- 语法：**assign <delay> net_value=expression;**
- **assign**语句一般适合对组合逻辑进行赋值，始终处于激活状态；称为连续赋值语句。
- 多条**assign**语句同时运行，并行赋值；与硬件执行过程一致。
- 例**3**：**1**位全加器的设计。




```
module  adder1(sum,cout,ina,inb,cin);
    input     ina,inb,cin;
    output    sum,cout;
    wire      sum,cout;
    assign    {cout,sum} = ina+inb+cin;
endmodule
```

（2）数据流描述方式：用assign语句建模

– **assign语句建模，例4:**




```
module  eg4(x1,x2,x3,x4,f,g,h);
    input    x1,x2,x3,x4;
    output   f, g, h;

    assign   g=(x1 & x3) | (x2 & x4);
    assign   h=(x1|~x3) & (~x2 | x4);
    assign   f = g | h;

endmodule
```

5. FPGA及HDL语言介绍

（3）行为描述方式：用always块语句建模

- 行为描述是对设计电路的逻辑功能的描述，表示输入和输出之间转换的行为，并不用关心设计电路使用哪些元件以及这些元件之间的连接关系。
- 语法：**always@（sensitivity statements）**

  < begin >

  procedural assignment statements；

  < end >

5. FPGA及HDL语言介绍

（3）行为描述方式：用always块语句建模

- 功能：当敏感列表中信号发生变化，块中语句就以顺序执行一次
- 关键字begin和end用于将多条语句组成顺序块（非阻塞赋值语句除外），一个Verilog模块中的多个always块之间并行关系。
- always块可以产生各种逻辑，电平触发的always块常用来描述组合逻辑行为；沿触发的always块常用于时序逻辑的功能描述。通过综合工具自动将其转换为相应的逻辑结构。

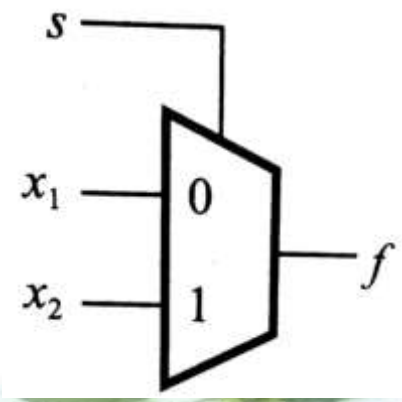
（4）行为描述方式：用always块语句建模

– always语句建模，例5：

```
//二选一数据选择器
module  eg5(x1,x2,s,f);
  input     x1,x2,s;
  output    f;
  reg f;
  always@(x1 or x2 or s)
     if(s==0)
        f=x1;
     else
        f=x2;
endmodule
```

```
//二选一数据选择器
module  eg5(input x1, x2,s,output reg f);

   always@(x1, x2, s)
      if(s==0)
         f=x1;
      else
         f=x2;
endmodule
```



5. FPGA及HDL语言介绍

（4）行为描述方式：用initial块语句建模

– initial语句语法格式为：

```
initial
begin
  语句1;
  …
end
```

– initial语句的使用次数也是不受限制的，但块内的语句仅执行一次，因此initial语句常用于仿真中的初始化。

5. FPGA及HDL语言介绍

- 一个程序可以只包含一个module，也可以包括多个module；对于较大规模的逻辑电路设计可以采用层次化设计方案。
- 例6:

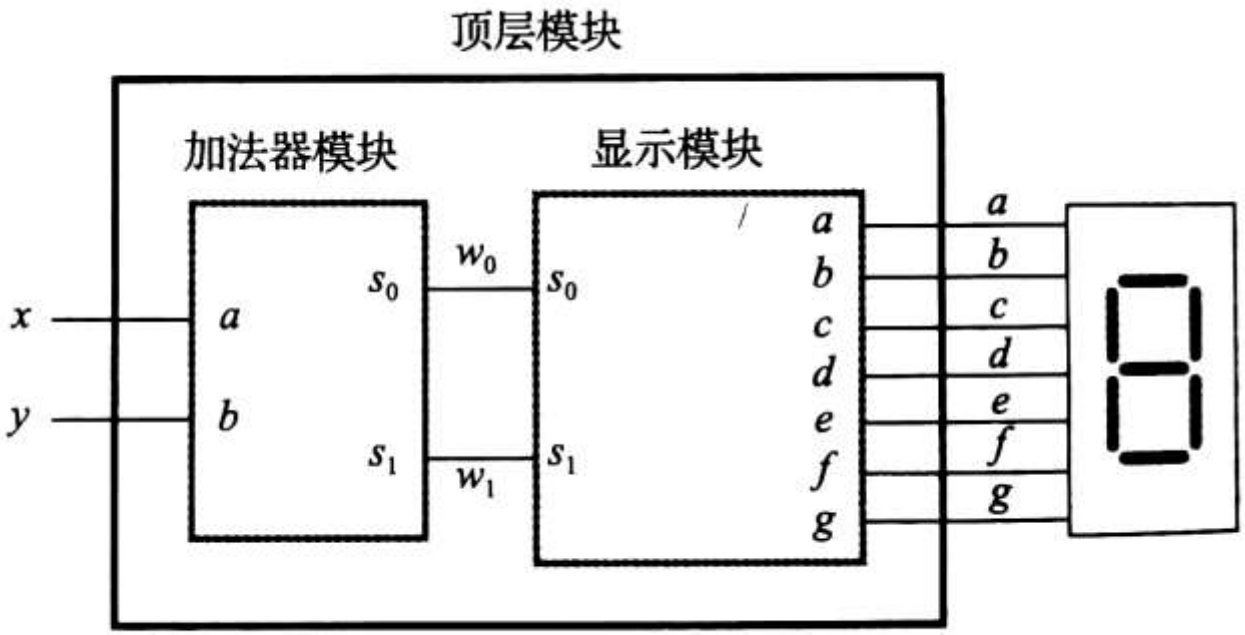


图 2.44　带有两个模块的逻辑电路

5. FPGA及HDL语言介绍

- **模块实例化方法1：位置关联法**

eg：bottom inst(i_A, i_B, o_And, o_Or, o_Not);

- bottom是之前写好的底层模块（必须与模块名一致），inst是实例化的模块名（可以自定义）。
- 括号中的信号分别按顺序与模块bottom中的信号对应，如**i_A**对应**iA**，对应信号名可以不同。

5. FPGA及HDL语言介绍

- **模块实例化方法2：名称关联法（推荐）**

```
eg：  bottom inst(.iA  (i_A),
                  .iB  (i_B),
                  .oAnd  (o_And),
                  .oOr  (o_Or),
                  .oNot  (o_Not)
                  );
```

  - **Top 模块中的i_A与 bottom 模块中iA对应；**
  - **几个信号顺序可以互换。**

5. FPGA及HDL语言介绍

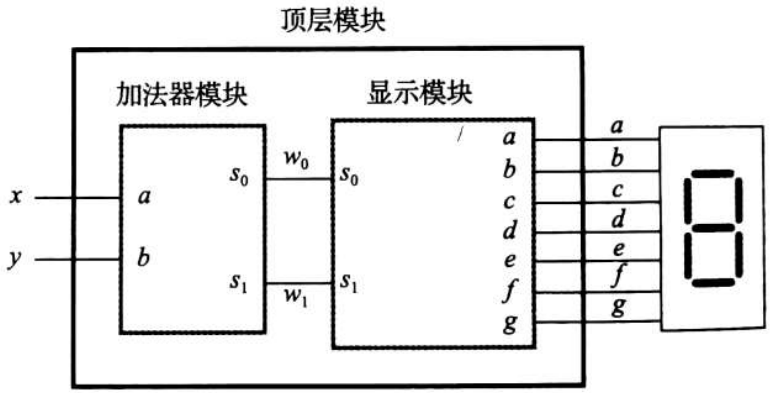


图 2.44　带有两个模块的逻辑电路

```
//加法器模块
module  adder(
    input     a,b,
    output    s1,s0
);
    assign    s1=a&b;
    assign    s0=a^b;
endmodule
```

```
//七段显示驱动模块
module  display(
    input     s1,s0,
    output    a, b, c, d, e, f, g
);
    assign    a=~s0;
    assign    b=1;
    assign    c=~s1;
    assign    d=~s0;
    assign    e=~s0;
    assign    f=~s1 & ~s0;
    assign    g=s1 & ~s0;
endmodule
```

5. FPGA及HDL语言介绍

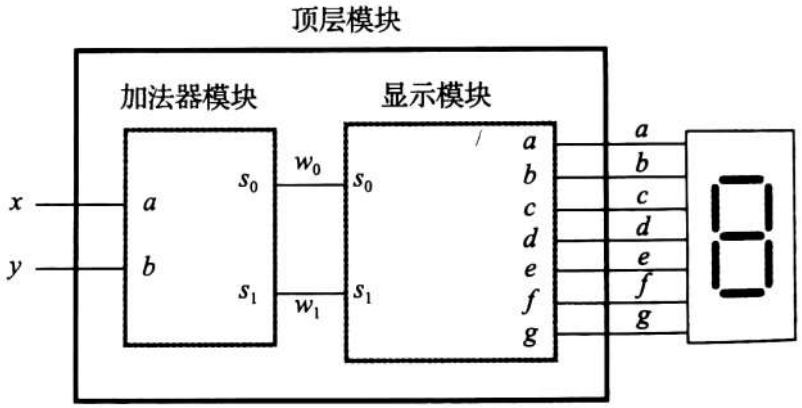


图 2.44　带有两个模块的逻辑电路

```
//顶层模块
module  adder_display(
    input   wire  x, y,
    output  wire  a, b, c, d, e, f, g
);
    wire    w1,w0;
```

```
//采用了名称关联法
adder U1(
.a (x),
.b (y),
.s1 (w1),
.s0 (w0)
);
//采用了位置关联法
display U2(w1,w0,a,b,c,d,e,f,g);

endmodule
```

5. FPGA及HDL语言介绍

实验板使用注意事项

- 插拔USB数据线前，电源开关处于off状态。
- Micro USB避免插反，应该是带勾的平头朝上，如果发现很难插拔，需要检查是否插反。（损坏概率最高）
- 尽量避免湿手、带静电的手接触电路板及外设接口部分。
- 用外设的时候注意电平，应该是3.3V兼容5V输入，避免高电压接入。
- 注意安装Vivado和工程目录不要出现中文，也不要使得工程所在目录太深。
- 电脑的用户名不要设置为中文，会出现各种诡异的问题。

5. FPGA及HDL语言介绍

- 三种描述方式（结构型、数据流、行为级）
  - 初学时，不要一次填加三个“ .v ”源文件（可能会综合出错）。
  - 可以采用注释方式，依次运行；如：运行结构型时，用/* */注释掉其他两种方式。
- 常见的报错问题
  - 综合报错：标点符号出错；约束文件出错；目录非中文；
  - 仿真报错：模块例化出错；标点符号出错；

5. FPGA及HDL语言介绍

◆ 主要设计流程及注意事项：

- .V输入（add Sources）->综合（Run synthesis）->添加约束文件->添加仿真文件->功能仿真->运行实现过程（Run implementation）->生成bit流(Generate Bitstream)->Auto connect target ->Program Divice
- 区分仿真和综合：功能仿真不是必须，但却非常必要；一定要学会通过看仿真时序来验证电路功能的正确性。
- 习惯查看综合后的设计，如RTL Schematic，熟悉“什么样的代码会综合成什么样的电路”，有助于编程技能提升。
- 生成bit流后，再接入电路板；

5. FPGA及HDL语言介绍

◆ 关于Testbench代码（测试文件代码）：

- 结构

module testbench；// 通常testbench没有输入与输出端口

信号或变量定义声明；

使用initial或always语句来产生激励波形；

例化设计模块；

监控和比较输出响应；

endmodule

5. FPGA及HDL语言介绍

◆ 关于Testbench代码：

- 在testbench的开头定义时间单位和仿真精度，比如：`timescale 1ns/1ps，前一个是代表时间单位，后一个代表仿真时间精度。
- # 符号是指延迟相应的时间单位，该时间单位由timscale决定。
- 仿真代码中，输出一定为wire型，输入变量通常为reg型；initial里放重置信号，always里放时钟信号。
- 如果initial块语句过于复杂，可以考虑将其分为互不相干的几部分，用数个initial块来描述。在仿真时，这些initial块会并发运行，方便阅读和修改。如对iA和iB并行产生激励数据。